Dissolved Oxygen Pipeline

Introduction

This vignette walks through the complete dissolved oxygen (DO) processing pipeline using the `waterlogged` package. We use real test data from the COOK_WE_40 wetland station as our example.

The pipeline processes raw HOBO U26 DO logger data through these steps:

1. Read and quality-check barometric (air pressure) data
2. Read raw dissolved oxygen logger files
3. Attach barometric pressure to the DO data
4. Convert DO concentration (mg/L) to percent saturation (%sat)
5. Run automatic quality checks on the DO data
6. Correct sensor drift using field reference measurements
7. Make manual corrections (ice, dry, disturbance) as needed
8. Export the final clean dataset

Why barometric data matters for DO: Dissolved oxygen saturation depends on atmospheric pressure. A DO reading of 8 mg/L at sea level means something different than 8 mg/L at high elevation. By attaching barometric pressure, we can convert the raw mg/L reading into percent saturation (%sat) — a measure that accounts for altitude and weather conditions.

Why drift correction matters: DO sensors gradually lose accuracy over time as the sensing element degrades ("fouling"). Field reference measurements taken during site visits tell us how far the sensor has drifted. The drift correction adjusts the logged data so it matches the field references.

This vignette assumes you have already installed `waterlogged` from the develop branch:

```
install.packages("pak")
pak::pkg_install("dfo-pacific-science/waterlogged@develop")
```

Setup

```
library(waterlogged)
```

Define paths to the test data shipped with the package.

```
metadata_file  <- system.file("extdata/dissolved_oxygen/metadata.csv", package = "waterlogged")
baro_folder    <- system.file("extdata/dissolved_oxygen/baro", package = "waterlogged")
do_folder      <- system.file("extdata/dissolved_oxygen/DO", package = "waterlogged")
reference_file <- system.file("extdata/dissolved_oxygen/field_references.csv", package = "waterlogged")
```

Step 1: Bind and QAQC Barometric Data

Reads all barometric logger CSVs, combines them, and runs quality checks (out-of-range, spikes, flatlines).

We need clean barometric data to convert DO from mg/L to %sat. If the baro data has errors, they'll contaminate the %sat values. This is the same process used in the water level pipeline — baro data is the foundation for both.

```
baro_bound <- bind_hobo_files(
  path_to_raw_folder = baro_folder,
  metadata_path      = metadata_file
)

baro_data <- baro_bound[[1]]

cat("Baro data:", nrow(baro_data), "rows,",
    length(unique(baro_data$site_station_code)), "stations\n")
#> Baro data: 39589 rows, 3 stations
```

```
baro_qc <- barometric_qaqc_all(
  baro_data         = baro_data,
  metadata_path     = metadata_file,
  pressure_low_kpa  = 85,
  pressure_high_kpa = 105,
  temp_low_limit    = -40,
  temp_high_limit   = 50,
  spike_threshold_kpa = 1.5,
  flatline_n        = 6
)

cat("Baro QAQC complete:", nrow(baro_qc), "rows\n")
#> Baro QAQC complete: 39589 rows
cat("  Spikes flagged: ", sum(baro_qc$edit_spike, na.rm = TRUE), "\n")
#>   Spikes flagged:  0
cat("  Flatlines:      ", sum(baro_qc$edit_flatline, na.rm = TRUE), "\n")
#>   Flatlines:       0
```

Step 2: Bind Raw Dissolved Oxygen Logger Files (v0.1)

Reads the raw HOBO U26 DO logger CSV and links it to the metadata.

The HOBO U26 records two channels: DO concentration in mg/L and water temperature in °C. This step parses them into standard column names: `do_mgl` and `watertemp_C`.

About the COOK_WE_40 data:

- Logger: HOBO U26 (SN 21473863)
- Deployed: April 14 to October 21, 2025
- Location: COOK wetland, station WE_40
- One download file covering the full deployment

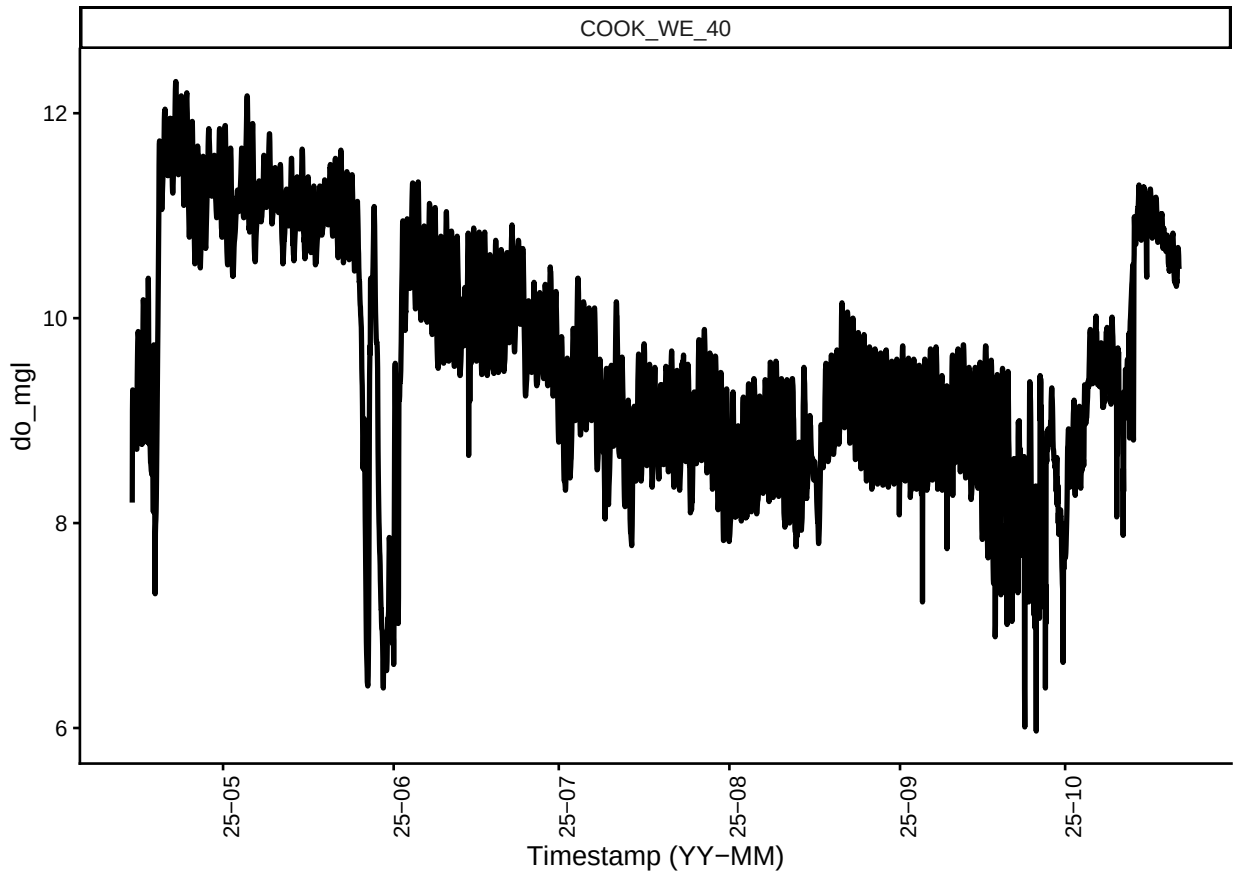
```
do_bound <- bind_hobo_files(
  path_to_raw_folder = do_folder,
  metadata_path      = metadata_file
```

```
)

do_data <- do_bound[[1]]
do_plot <- do_bound[[2]]

cat("DO data:", nrow(do_data), "rows\n")
#> DO data: 4566 rows
cat("Stations:", paste(unique(do_data$site_station_code), collapse = ", "), "\n")
#> Stations: COOK_WE_40
cat("Date range:", as.character(min(do_data$timestamp)),
    "to", as.character(max(do_data$timestamp)), "\n")
#> Date range: 2025-04-14 12:00:00 to 2025-10-21 17:00:00
cat("DO range:", round(min(do_data$do_mgl, na.rm = TRUE), 2), "to",
    round(max(do_data$do_mgl, na.rm = TRUE), 2), "mg/L\n")
#> DO range: 5.97 to 12.31 mg/L
```

```
do_plot
```



Step 3: Attach Barometric Data (v0.2)

Finds the nearest barometric station to COOK_WE_40 and joins the air pressure and temperature readings to each DO timestamp.

We need atmospheric pressure at each timestamp to:

1. Convert DO from mg/L to percent saturation
2. Help detect dry-sensor conditions (comparing air and water temp)

You can select the baro station in two ways: `"auto"` (calculates distance and picks nearest) or by specifying a station code directly. Here we use `DARL_ST_30` directly, since it's the closest baro station to COOK_WE_40 and provides full temporal coverage.

```
do_with_baro <- add_nearest_baro(
  input_data          = do_data,
  baro_data           = baro_qc,
  metric              = "DO",
  baro_site_selection = "DARL_ST_30",
  metadata_path       = metadata_file
)

do_data <- do_with_baro[[1]]

cat("After baro attach:", nrow(do_data), "rows\n")
#> After baro attach: 4566 rows
cat("Baro station(s) used:",
    paste(unique(na.omit(do_data$baro_site_stn_code)), collapse = ", "), "\n")
#> Baro station(s) used: DARL_ST_30

baro_na <- sum(is.na(do_data$airpress_kPa))
cat("Missing baro data:", baro_na, "of", nrow(do_data), "rows",
    paste0("(", round(baro_na / nrow(do_data) * 100, 1), "%)"), "\n")
#> Missing baro data: 1 of 4566 rows (0%)
```

Step 4: Convert DO mg/L to Percent Saturation (v0.3)

Calculates percent air saturation (%sat) from the DO concentration, water temperature, and atmospheric pressure.

Percent saturation tells you how much oxygen is in the water relative to what the water could hold at that temperature and pressure. It's a better indicator of ecosystem health than mg/L alone because it accounts for physical conditions. For example, cold water holds more oxygen than warm water. So 8 mg/L in cold water might be only 70% saturated, while 8 mg/L in warm water might be 95% saturated.

The math uses the Weiss (1970/1974) oxygen solubility equation to calculate the theoretical maximum DO, then divides the measured DO by that maximum and multiplies by 100.

```
do_data <- convert_do_mgl_percsat(do_data)

cat("DO %sat range:", round(min(do_data$do_percsat, na.rm = TRUE), 1), "to",
    round(max(do_data$do_percsat, na.rm = TRUE), 1), "%\n")
#> DO %sat range: 52 to 105.8 %
```

```
head(do_data[, c("timestamp", "do_mgl", "watertemp_C",
                 "airpress_kPa", "do_percsat")])
#> # A tibble: 6 x 5
#>   timestamp           do_mgl watertemp_C airpress_kPa do_percsat
#>   <dttm>               <dbl>       <dbl>        <dbl>      <dbl>
#> 1 2025-04-14 12:00:00   8.2         18.9         97.3         92
```

```
#> 2 2025-04-14 13:00:00   9.1         18.2         97.2        101.
#> 3 2025-04-14 14:00:00   9.3         18.0         97.1        102.
#> 4 2025-04-14 15:00:00   9.26        18.0         97.0        102.
#> 5 2025-04-14 16:00:00   9.24        18.2         96.9        102.
#> 6 2025-04-14 17:00:00   9.12        18.6         96.9        102
```

Step 5: Automatic DO QAQC

Runs automatic quality checks on the dissolved oxygen data and creates adjusted columns with problematic values corrected or removed.

What it checks:

1. **Logger errors** — the code −888.88 means the sensor malfunctioned
2. **Negative DO** — DO between 0 and −1 mg/L corrected to 0; below −1 mg/L removed
3. **High outliers** — DO above 21 mg/L removed (physically implausible)
4. **Ice detection** — water temp < 0.3 °C flagged; -1 °C removed
5. **Dry detection** — a 12-point sliding window checks if %sat 100 AND air-water temp diff 2 °C (sensor in air)

Adjusted columns created: `do_mgl_adj`, `do_percsat_adj`, `watertemp_C_do_adj`.

The same flag vs edit pattern applies: `flag_*` = "please review" (data unchanged), `edit_*` = "we changed the data".

```
do_qc <- dissox_qaqc(
  input_data     = do_data,
  select_station = "COOK_WE_40"
)

cat("QAQC results for COOK_WE_40:\n")
#> QAQC results for COOK_WE_40:
cat("  Total rows:       ", nrow(do_qc), "\n")
#>   Total rows:        4566
cat("  Logger errors:    ", sum(do_qc$do_error, na.rm = TRUE), "\n")
#>   Logger errors:     0
cat("  DO < -1 removed:  ", sum(do_qc$do_neg_lt_minus1, na.rm = TRUE), "\n")
#>   DO < -1 removed:   0
cat("  DO -1 to 0 fixed: ", sum(do_qc$do_neg_between, na.rm = TRUE), "\n")
#>   DO -1 to 0 fixed:  0
cat("  DO > 21 removed:  ", sum(do_qc$do_high, na.rm = TRUE), "\n")
#>   DO > 21 removed:   0
cat("  Ice flagged:      ", sum(do_qc$temp_flag_ice, na.rm = TRUE), "\n")
#>   Ice flagged:       0
cat("  Dry flagged:      ", sum(do_qc$flag_do_dry, na.rm = TRUE), "\n")
#>   Dry flagged:       0
```

```
qaqc_log <- attr(do_qc, "qaqc_log")
if (!is.null(qaqc_log) && nrow(qaqc_log) > 0) {
  cat("QAQC log entries:\n")
  print(qaqc_log[, c("field", "action", "code", "ts_start", "ts_end")])
}
```

```
#> QAQC log entries:
#> # A tibble: 1 x 5
#>   field      action    code  ts_start ts_end
#>   <chr>      <chr>     <chr> <chr>    <chr>
#> 1 do_mgl_adj AUTO_QAQC <NA>  <NA>     <NA>
```

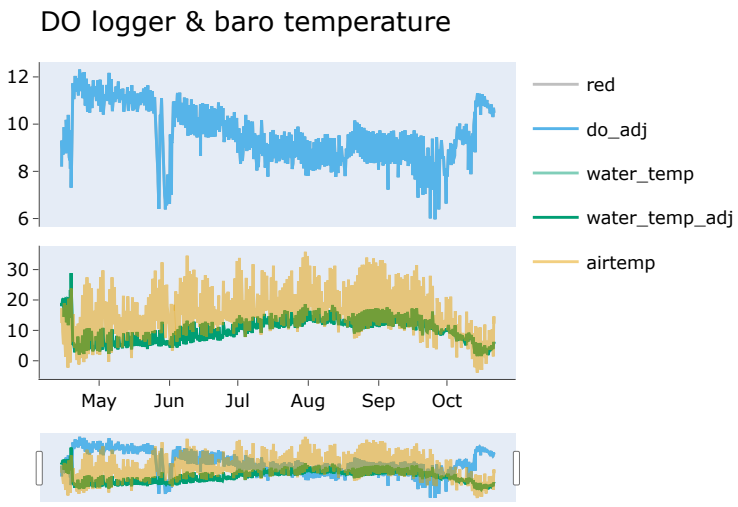
Step 6: Visual Review

Creates an interactive multi-panel plot for reviewing the DO data. Before applying drift correction or manual adjustments, you need to see the data.

How to read the plot:

- Light traces = raw values (original, unmodified)
- Dark traces = adjusted values (after auto QAQC)
- Colored dots = flag locations (ice, dry, error)
- Use the range slider at the bottom to zoom into specific periods
- Hover over points for exact values

```
plot_qaqc_timeseries(
  do_data        = do_qc,
  baro_data      = baro_qc,
  select_station = "COOK_WE_40"
)
```


DO logger & baro temperature
12
10
8
6
30
20
10
0
May
Jun
Jul
Aug
Sep
Oct
red
do_adj
water_temp
water_temp_adj
airtemp

Step 7: Drift Correction

Adjusts the logged DO data to match field reference measurements, correcting for gradual sensor drift between site visits.

DO sensors slowly lose accuracy over their deployment. The sensing membrane gets fouled (coated with biofilm, sediment, etc.), causing readings to drift away from true values. During field visits, a handheld reference instrument (e.g., a calibrated YSI) provides a "truth" measurement. Drift correction uses these reference points to straighten out the logged data.

The reference file contains discrete field measurements with an `event_type` for each:

- **"calibration"** — the sensor was recalibrated during this visit. The function corrects data before the calibration and trusts data after it.
- **"check"** — the reference was measured but the sensor was NOT touched. Used to quantify drift at that point in time.
- **"ignore"** — skip this measurement.

Drift correction methods:

- **"ratio"** (default): Multiplicative correction. Better for sensors where drift is proportional (typical for DO sensors).
- **"offset"**: Additive correction. Better when drift is a constant shift.

```
ref_data <- read.csv(reference_file)
ref_data[ref_data$metric == "do", ]
#>   metric site_station_code       timestamp watertemp_C value    unit
#> 5     do        COOK_WE_40   5/2/2025 2:22          15    99 percsat
#> 6     do        COOK_WE_40  7/15/2025 4:12          15    98 percsat
#> 7     do        COOK_WE_40 7/20/2025 10:45          15    87 percsat
#> 8     do        COOK_WE_40 9/21/2025 12:00          15   100 percsat
#>    event_type calibration_datetime comments
#> 5 calibration        5/2/2025 2:00       NA
#> 6      ignore                            NA
#> 7       check                            NA
#> 8       check                            NA
```

```
do_qc <- wq_drift_correction(
  input_data     = do_qc,
  ref_data_path  = reference_file,
  select_station = "COOK_WE_40",
  metric         = "do",
  method         = "ratio",
  max_ref_gap_h  = 1
)

if ("edit_do_drift" %in% names(do_qc)) {
  drift_count <- sum(do_qc$edit_do_drift, na.rm = TRUE)
  cat("Rows corrected by drift:", drift_count, "of", nrow(do_qc), "\n")
} else {
  cat("No drift correction column found - check reference data alignment.\n")
}
#> Rows corrected by drift: 3838 of 4566
```

```
drift_log <- attr(do_qc, "qaqc_log")
if (!is.null(drift_log) && nrow(drift_log) > 0) {
  cat("Drift correction log:\n")
  print(drift_log)
}
#> Drift correction log:
#> # A tibble: 1 x 13
#>   station  metric field action code  action_note manual_note ts_start
#>   <chr>    <chr>  <chr> <chr>  <chr> <chr>       <chr>       <dttm>
#> 1 COOK_WE~ DO     do_p~ DRIFT~ DRIF~ Forward li~ AUTOMATIC ~ 2025-04-14 13:00:00
#> # i 5 more variables: ts_end <dttm>, duration_hours <dbl>, fun_name <chr>,
#> #   run_at <dttm>, user <chr>
```

Step 8: Manual Adjustments (Examples)

This section demonstrates how to manually remove bad data. In a real project, you would apply these only after reviewing the plot and identifying specific problem periods.

For DO data, the three reasons for removal are:

- **"ice"** — sensor frozen in ice; DO readings are unreliable. Sets `do_mgl_adj`, `do_percsat_adj`, `watertemp_C_adj` to `NA`.
- **"dry"** — sensor exposed to air (water level dropped below it). Same columns set to `NA`.
- **"disturbance"** — physical disturbance (e.g., debris, maintenance). Sets DO columns to `NA` but can keep temperature with `keep_temp = TRUE`.

Each function returns a new data frame. Always assign the result back to your working variable so corrections accumulate.

Remove a dry period

```
do_qc <- adjust_logger_NA(
  input_data       = do_qc,
  select_station   = "COOK_WE_40",
  metric           = "dissolvedoxygen",
  reason_to_adjust = "dry",
  timestamp_start  = "2025-08-01 00:00:00",
  timestamp_end    = "2025-08-15 00:00:00",
  manual_note      = "Sensor exposed during low-flow period"
)
```

Remove an ice period

```
do_qc <- adjust_logger_NA(
  input_data       = do_qc,
  select_station   = "COOK_WE_40",
  metric           = "dissolvedoxygen",
  reason_to_adjust = "ice",
  timestamp_start  = "2025-01-01 00:00:00",
```

```
  timestamp_end    = "2025-03-15 00:00:00",
  manual_note      = "Ice confirmed from water temperature data"
)
```

Remove a disturbance (keep temperature)

Sometimes you want to remove DO readings but keep the temperature data (e.g., during a brief maintenance event where the temp sensor was fine).

```
do_qc <- adjust_logger_NA(
  input_data       = do_qc,
  select_station   = "COOK_WE_40",
  metric           = "dissolvedoxygen",
  reason_to_adjust = "disturbance",
  timestamp_start  = "2025-06-01 12:00:00",
  timestamp_end    = "2025-06-01 14:00:00",
  keep_temp        = TRUE,
  manual_note      = "Brief logger maintenance - DO affected, temp OK"
)
```

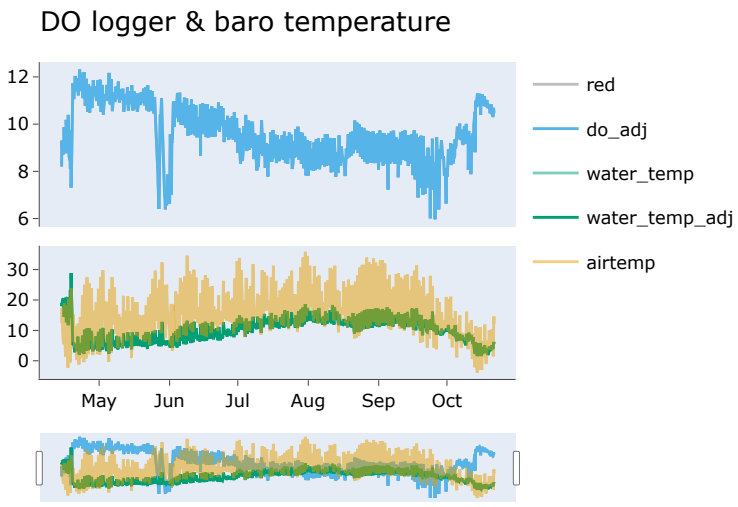
After any manual adjustments, always re-plot to verify:

```
plot_qaqc_timeseries(
  do_data        = do_qc,
  baro_data      = baro_qc,
  select_station = "COOK_WE_40"
)
```

Step 9: Final Review

Re-plot the data after all corrections to verify the final result. The adjusted traces should look reasonable with no obvious artifacts. Compare raw (light) vs adjusted (dark) to see exactly what changed.

```
plot_qaqc_timeseries(
  do_data        = do_qc,
  baro_data      = baro_qc,
  select_station = "COOK_WE_40"
)
```


DO logger & baro temperature
12
10
8
6
30
20
10
0
May
Jun
Jul
Aug
Sep
Oct
red
do_adj
water_temp
water_temp_adj
airtemp

Step 10: Export Clean Data (v1.0)

Writes the final cleaned data to CSV files, one per station per year.

What gets exported:

- `site_station_code`, `timestamp` (identifiers)
- `do_mgl`, `do_mgl_adj` (raw and adjusted DO in mg/L)
- `do_percsat`, `do_percsat_adj` (raw and adjusted DO in %sat)
- `watertemp_C`, `watertemp_C_do_adj` (raw and adjusted water temperature)
- All `flag_*` and `edit_*` columns (the full audit trail)

```
cat("Final dataset:", nrow(do_qc), "rows x", ncol(do_qc), "columns\n\n")
#> Final dataset: 4566 rows x 39 columns

cat("Summary of adjusted DO (%sat):\n")
#> Summary of adjusted DO (%sat):
print(summary(do_qc$do_percsat_adj))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#>   50.67   83.45   90.65   91.78   99.18  134.00       1

cat("\nSummary of adjusted DO (mg/L):\n")
#>
#> Summary of adjusted DO (mg/L):
print(summary(do_qc$do_mgl_adj))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   5.970   8.650   9.430   9.555  10.580  12.310
```

```
write_clean_data(
  input_data            = do_qc,
  path_to_output_folder = "output",
  metric                = "dissolvedoxygen"
)
```

Recap

Step	Function	Result
1	`barometric_qaqc_all()`	`baro_qc` (cleaned baro)
2	`bind_hobo_files()` on DO	`do_data` (raw DO)
3	`add_nearest_baro()`	`do_data` (DO + baro attached)
4	`convert_do_mgl_percsat()`	`do_data` (adds %sat)
5	`dissox_qaqc()`	`do_qc` (auto QAQC'd)
6	`plot_qaqc_timeseries()`	(visual review)
7	`wq_drift_correction()`	`do_qc` (drift corrected)
8	Manual adjustments	`do_qc` (updated)
9	`plot_qaqc_timeseries()`	(final review)
10	`write_clean_data()`	CSV files on disk

Key columns:

- `do_mgl` — raw DO in mg/L (never modified)

- `do_mgl_adj` — adjusted DO in mg/L (all corrections applied)
- `do_percsat` — raw DO in percent saturation (never modified)
- `do_percsat_adj` — adjusted DO in %sat (all corrections applied)
- `watertemp_C` — raw water temperature (never modified)
- `watertemp_C_do_adj` — adjusted water temperature
- `flag_*` — "something looks off — review" (data not changed)
- `edit_*` — "data was changed" (set to NA or corrected)
- `edit_do_drift` — TRUE where drift correction was applied

Use `do_percsat_adj` for analysis — it incorporates all corrections including sensor drift.